

0.6.1. La même ligne, avec une boucle while — de base, ailleurs

Le 0.6 sur la ligne 5 : un seul nombre change dans la boucle.

1. Ce qu'il faut comprendre

C'est le 0.6, avec un seul changement : la **ligne** : 5 au lieu de 0, dans le `poser` de la boucle.

Un seul nombre à changer, au lieu de vingt au 0.4.1 : la boucle écrit la ligne entière, et la ligne n'est dite qu'une fois.

C'est la version de **base** : une seule chose, à la deuxième place. Le 0.6.2 met les deux ensemble.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1334 octets** de cartouche.

1. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

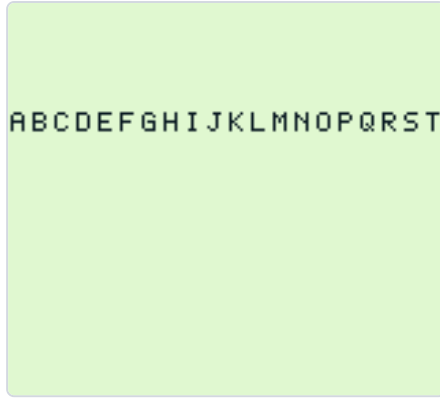
```
int main() {  
    // Le changement : la ligne, 0 → 5.  
    uint8_t i = 0;           // on part de la colonne 0  
    while (i < 20) {         // tant qu'on est dans l'écran  
        poser(i, 5, ALPHABET[i]); // la lettre n° i, colonne i, ligne 5  
        i++;                 // la suivante  
    }  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

A à T sur la ligne 5.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ la ligne 5 : A à T

Ensuite : **0.6.2. La même ligne, avec une boucle while — doublé, deux positions** — niveau 0 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++